

ARI: 自主研究基础设施

树神宇德

v0.8.1, 2026-06-01

摘要

本报告描述了一个自主研究基础设施 **ARI**, 它将基于树搜索的探索循环与论文生成流水线相结合。探索循环是对研究步节点的最佳优先树搜索 (BFTS); 节点扩展由 Reason-and-Act (ReAct [1]) 风格的智能体执行, 该智能体向 14 个领域技能的注册表发出工具调用。论文流水线将存活的谱系汇编为草稿, 并在层次化评分细则下迭代改稿。所有产物都驻留在单一的检查点目录中, 该目录是可复现性、谱系与成本核算的单位。本报告形式化算法与设计依据, 报告初步观察, 并讨论制约设计的确定性与新颖性的权衡。

1 引言

1.1 动机

机器学习及相邻领域的研究流程具有共同骨架: 构思研究构想、设计并执行实验、分析结果、撰写论文, 然后迭代。其中每一步对大语言模型而言都日益可行, 但将它们串接为单一自主循环至今仍是一个未解的工程问题。最接近此目标的已发表系统 — AIDE、AI Scientist 系列、VirSci 与 PaperBench 评估器 — 各自仅覆盖其中一两个轴向, 其余仍依赖人工介入。

ARI 旨在填补此空缺。我们假定用户提供研究问题与算力预算; 系统返回可发表的论文、支撑每项主张的实验代码, 以及完整的执行审计轨迹。该审计轨迹不可妥协: 搜索树的每一个节点、每一次工具调用、每一次模型调用, 以及每一美元的 API 花费, 均记录于单一的检查点目录之中 ([2] 采用了类似的纪律)。检查点是可复现性的单位。

1.2 贡献

本报告对以下三项贡献加以形式化:

1. 在研究步节点上的**最佳优先树搜索**, 其选择评分 $U(n)$ 将经验奖励、新颖性项与深度惩罚分离开来 (section 3.1)。
2. 由层次化评分细则 R 驱动的**论文生成流水线**; 其叶判官对各项主张进行评分, 系统评分为 $\text{score}(P) = \sum_i w_i \text{judge}_i(P)$ (section 4.3)。PaperBench 复现器作为工具调用集成 (section 4.5)。
3. **检查点作用域纪律**, 使流水线完全可复现: 每一项外部状态 (记忆、成本账本、谱系指针) 都写入

`$ARI_CHECKPOINT_DIR` 之下, 绝不进入用户主目录。这是确定性设计原则的运维落地 (section 2.5 完整列出五项原则; 本报告中将该原则称为 P2)。

1.3 本报告的范围

本报告描述 `ari-core` 与十四个 `ari-skill-*` 包的 v0.8.0 版本。v0.8.0 将 PaperBench 的再现路径提升为协议忠实的三阶段契约——智能体 rollout、再现与评分——增设了阻止 rollout 智能体臆断宿主未提供能力的宿主真实环境护栏, 使搜索评估层可配置, 并对一个真实的非可逆压缩对象实施了首次操作性再现阶段 (section 5)。本文以模块层级描述算法与数据流; 系统所用的 LLM 提示词逐字载录于 section B。本版本的经验研究尚属初步——section 5 报告迄今可得的观察, 受控基准留待今后工作。

1.4 结构

Section 2 自顶向下概述 ARI: 编排器、十四个技能、流水线 DAG 与设计原则。Sections 3 and 4 是两个技术核心, 各自含形式化定义与经验观察。Section 5 报告初步观察; section 6 定位本工作于先行研究中; section 7 汇集开放问题。

2 系统概览

2.1 完整堆栈

Figure 1 描绘了 v0.8.0 的 ARI 完整堆栈。系统分为一个驱动器与多个工具提供方。驱动器 (`ari-core`) 拥有编排器、BFTS 引擎、谱系遍历

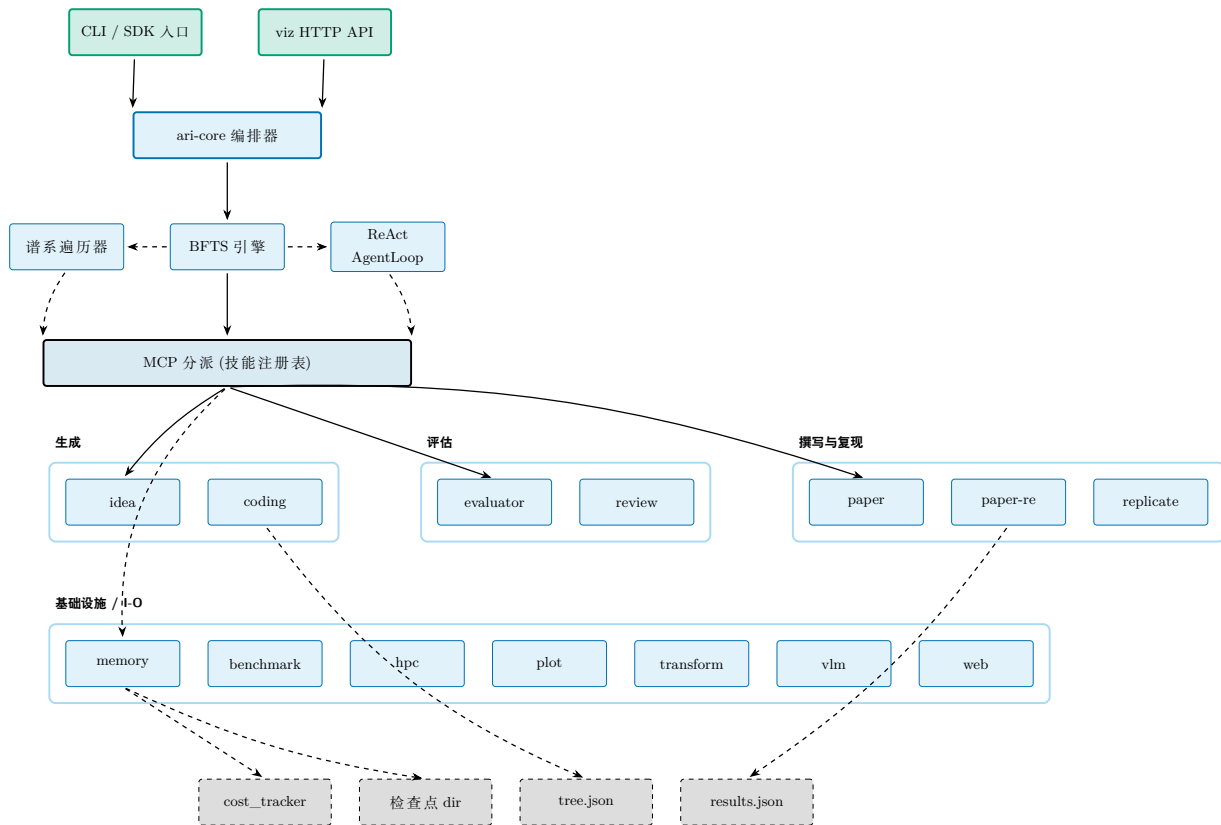


图 1: v0.8.0 时 ARI 的完整堆栈。CLI / SDK 入口驱动单一编排器 (ari-core); 编排器拥有三个引擎 (BFTS、谱系遍历器、ReAct AgentLoop), 并经 MCP 层向 14 技能注册表分派。所有持久化都落在单一检查点目录: 成本账本、tree.json、results.json。实线为控制, 虚线为状态。

器、智能体循环、检查点、成本追踪器与配置加载器。每个技能为独立 Python 包, 以模型上下文协议 (Model Context Protocol, MCP) 服务器形式发布, 并附自身 skill.yaml 声明。MCP 是 Anthropic 标准化的 JSON-RPC-over-stdio 接口, 用于将工具式能力暴露给 LLM 代理循环; ARI 以此作为 ari-core 与各技能之间的统一接缝。当前在树的 14 个技能: benchmark、coding、evaluator、hpc、idea、memory、orchestrator、paper、paper-re、plot、replicate、transform、vlm、web。

编排器只负责调用分派, 不执行领域工作。这种分离至关重要: 技能可被增删或替换而不必触动搜索算法——BFTS 引擎并不知道哪些技能参与运行; 一切由 MCP 分派层中介。

Figure 2 是本报告其余部分使用的「运营视图」(控制 + 状态输出); fig. 1 是「分层视图」(驱动器 → MCP → 技能 → 持久化)。

2.2 流水线 DAG

一次运行可归约为四个工位的流水线 (fig. 3)。idea 工位选取或生成研究问题; exploration 工位 BFTS 下扩展节点; paper 工位将最佳谱系汇编为草

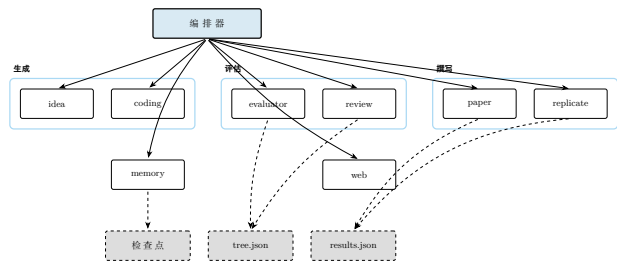


图 2: 运营视图: 编排器分派至技能注册表; 每个技能仅向当前活动的检查点目录写入。虚线表示状态。

稿; review 工位以评分细则评估草稿。该图为 DAG 加上一条由 review 回到 paper 的反向边: 仅此边可能引入非单调性 (改稿可能恶化某一轴), 而收敛标准 (section 4.4) 则是限定反向边数量的约束。

流水线由编排器中的单一运行器驱动; 科学数据组装器弥合探索输出与论文输入之间的间隙, 汇集撰写器所需的按节点产物。

2.3 BFTS 控制流

Figure 4 端到端追踪一次 BFTS 步骤。实现详见 section 3; 该图固定本报告其余部分使用的词汇 (前沿、选择、回退、expand、prune、停滞) 以及实

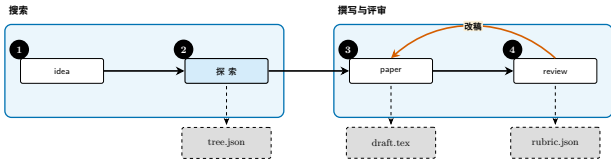


图 3: 流水线 DAG: idea $\rightarrow$ exploration $\rightarrow$ paper $\rightarrow$ review, 由 改稿 闭合反向边。虚线输出持久化至检查点; 改稿循环重写草稿并重跑评审。

现各 LLM 主导步骤的提示文件 (bfts_select.md、bfts_expand_select.md、bfts_expand.md)。

2.4 paper-re 组件视图

Figure 5 是 paper-re 的第二级视图。在此层 ARI 的贡献小但具体: 一层薄适配器包裹每个智能体工具以限制其输出大小, 并将 vendor 化求解器的硬编码路径重定向到 ARI 的按节点工作空间; 其余皆从 Paper-Bench 上游逐字流过, 使 ARI 与上游复现任务框架保持一致。该集成在 section 4.5 中详述。

2.5 检查点作用域与设计原则

ARI 围绕五条设计原则 (P1-P5) 构建。最具约束力者为 **P2: 确定性**。每一步随机化均记录其种子; 每次 LLM 调用均记录模型身份、提示与输出; 每次工具调用都以其节点身份标记 (写时复制护栏) 以确保写入落在所属节点的工作目录内。重跑检查点必须再现相同的产物, 字节级一致, 除非字段被显式标记为时钟相关或模型端非确定。

其余原则简述如下:

- **P1 单一产物树**: 运行的全部输出位于 `$ARI_CHECKPOINT_DIR` 之下, 不外溢至 `$HOME` 或 `/tmp`。
- **P3 小组件**: 每个技能为独立包; 替换技能不需重建 ari-core。
- **P4 显式谱系**: 每个节点与每个检查点均记录其父级, 派生实验仅须重算其差量。
- **P5 成本透明**: 成本追踪器为每次模型调用附加美元成本, 使运行预算可控。

检查点序列化器写出三个顶层文件: 完整搜索树写入 `tree.json`, 按节点的载荷写入 `nodes_tree.json`, 运行末聚合写入 `results.json`。跨检查点的谱系遍历由子向祖先攀登。

Figure 6 描绘检查点的磁盘形状: 左侧为共享运行级文件, 右侧为按节点工作目录。该形状即契约: 任

何把 ARI 输出当输入的工具 (后续运行、回归检查、论文草稿), 只需指向单一这样的目录即可。

3 探索算法

3.1 形式化

探索循环维护一棵树 $\mathcal{T} = (\mathcal{N}, E)$, 其中每个节点 $n \in \mathcal{N}$ 代表一次研究步骤: 构想修订、代码改动、基准运行、分析等。节点携带

- 离散状态 $s(n) \in \{\text{open}, \text{eval}, \text{done}, \text{failed}\}$,
- 取自 `NodeLabel` 枚举的类别标签,
- 用于按轴评估指标的自由形式 `dict node.metrics`, 含独立标量 `_scientific_score` (取值范围 $[0, 1]$),
- 标志节点是否携带实测值 (而非临时文本) 的布尔 `has_real_data`, 以及
- 选择提示与剪枝谓词使用的簿记字段 `depth`。ARI 不重试失败节点, 因此不会暴露任何 `retry_count` 信号。

将按轴信号汇总为标量 `scientific score` 的工作委托给任何实现 `Evaluator` 协议的对象; ARI 不固定线性组合。该协议是 BFTS 与下游评分细则评估之间唯一的接缝, 保持搜索引擎对如何将轴归约为标量保持中立。

Run-loop 状态. 外层循环每次迭代变换以下元组:

$$S = (\mathcal{F}, \mathcal{P}, e, H),$$

其中 $\mathcal{F} \subseteq \mathcal{N}$ 为可扩展的完成节点集合 (done / failed), $\mathcal{P} \subseteq \mathcal{N}$ 为处于 open 等待执行的节点集合, $e: \mathcal{N} \rightarrow \mathbb{N}$ 记录每个前沿节点被扩展过几次, $H = (l_1, \dots, l_t)$ 是已执行标签的滑动窗 (长度上限 $W = 20$, 详见 section 3.3)。初始状态为 $S_0 = (\emptyset, \{n_{\text{root}}\}, \mathbf{0}, ())$ 。

剪枝谓词. 硬性探索预算由如下谓词表达:

$$\Pi(n; |\mathcal{N}|) = \mathbb{K}[|\mathcal{N}| \geq B_N] \vee \mathbb{K}[\text{depth}(n) \geq B_D] \vee \sigma(n), \quad (1)$$

其中 $B_N = \text{config.max_total_nodes}$, $B_D = \text{config.max_depth}$, sterile 谓词

$$\sigma(n) = \mathbb{K}[|\Delta_n| = 0] \quad (2)$$

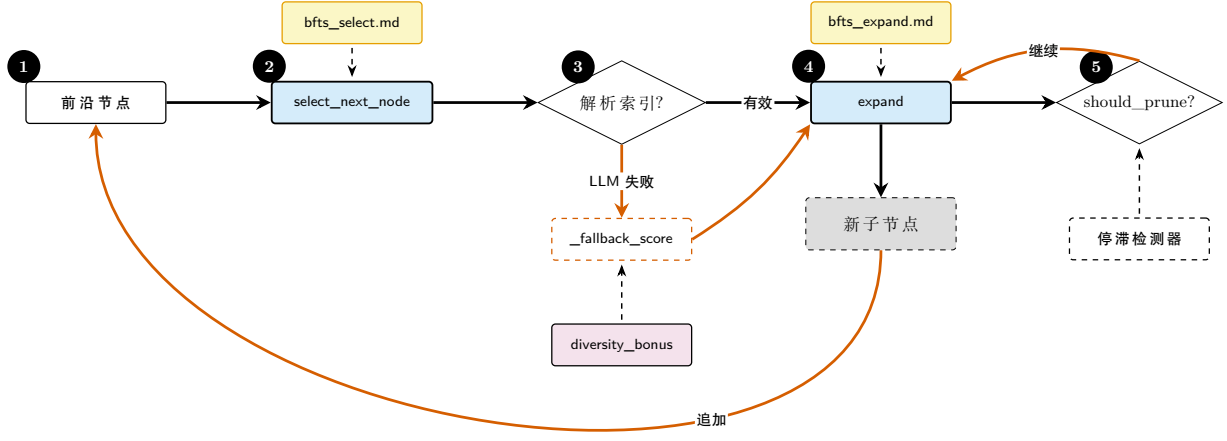


图 4: BFTS 控制流。两个 LLM 主导的决策点 (`select_next_node`、`expand`) 各自查询独立的提示文件; 选择决策在 LLM 返回不可解析索引时, 回退到确定性 `_fallback_score` (`_scientific_score` 与 `diversity_bonus` 之和)。剪枝由 `should_prune`(max-total-nodes 硬截止) 与停滞检测器共同把关。详见 section 3。

(Δ_n 为 n 新增、修改或删除的文件集合) 当节点在 agent loop 结束时相对父节点未写出任何新工件时为真。run loop 计算该文件差分, 并把结果作为布尔 `_sterile` 标志记录到节点上; `should_prune` 只读取该标志与两个上限。调用方每次迭代传入活动的 $|\mathcal{N}|$, 因此节点数有唯一真值源。步数与成本预算由调用侧的成本追踪器单独强制, 而不在 BFTS 引擎内。

Retire 谓词。 在 Π 之上, run-loop 还应用更软的前沿 *retire* 谓词 ρ , 在父节点 f 不再是最高产线索时将其从 $\mathcal{F}$ 中移除:

$$\rho(f, c) = \underbrace{\mathbb{I}[r(c) > r(f)]}_{\text{规则 A}} \vee \underbrace{\mathbb{I}[e(f) \geq E_{\max}]}_{\text{规则 B}}, \quad (3)$$

其中 $E_{\max} = \text{config.max_expansions_per_node}$ (默认 4)。规则 A 在子节点 c 一旦超越 f 时退役 f ; 规则 B 限制每个父节点的预算, 从而展开搜索面。 ρ 不删除节点, 其历史仍保留在 $\mathcal{T}$ 中, 仅撤销其继续被扩展的资格。

外层循环单次迭代的阶段分解 (内层扩展子循环、并行 ReAct 批次、执行后 retire) 见 fig. 7; 显式伪代码见 section 3.2 的 algorithm 1。

3.2 Run-loop 算法

algorithm 1 给出显式伪代码。内层 while 循环每次填入一个空 worker 槽位 (因此 worker 必在下次提出扩展前开始运行); 外层块通过反复调用 `select_next_node` (每次一个节点) 组装批次直至达到 worker 上限, 再并行执行该批次。执行后块 (a) 把

已执行标签追加到 H 以使多样性奖励反映真实执行, (b) 应用 ρ 的规则 A / B。

3.3 节点选择 (LLM 主导)

ARI 有意不将节点排序归约为闭式标量效用。相反, `select_next_node` (「下一智能体步该运行哪个 pending 节点」) 与 `select_best_to_expand` (「哪个完成节点最值得扩展」) 都把候选描述列表交给 LLM, 并解析返回的单个 0 起始索引——每次调用恰好选出一个节点。每个节点 n 的候选描述形如:

```
[i] id=..., depth=..., label=...,
has_real_data=..., metrics={...},
summary=...
```

运行选择侧 (`select_next_node`) 会对当前获得奖励的节点在行尾追加 `diversity_bonus=+0.05`; `select_best_to_expand` 不追加。消费它的提示分别于 section B.3 (选择) 与 section B.2 (扩展目标) 逐字载录。提示中列出的选择标准: `has_real_data=True` 且 `metrics` 强者高价值; `metrics` 整体多目标加权; 深而成果好的节点值得继续; `diversity_bonus` 仅当软性平手解决项处理。 `label` 字段与展示给 `select_best_to_expand` 的信息一致, 并且不存在误导性的“偏好低 retry”标准。

多样性奖励

多样性奖励 BFTS `diversity_bonus` 以 `_recent_label_history` (由 `record_run` 填充的滑

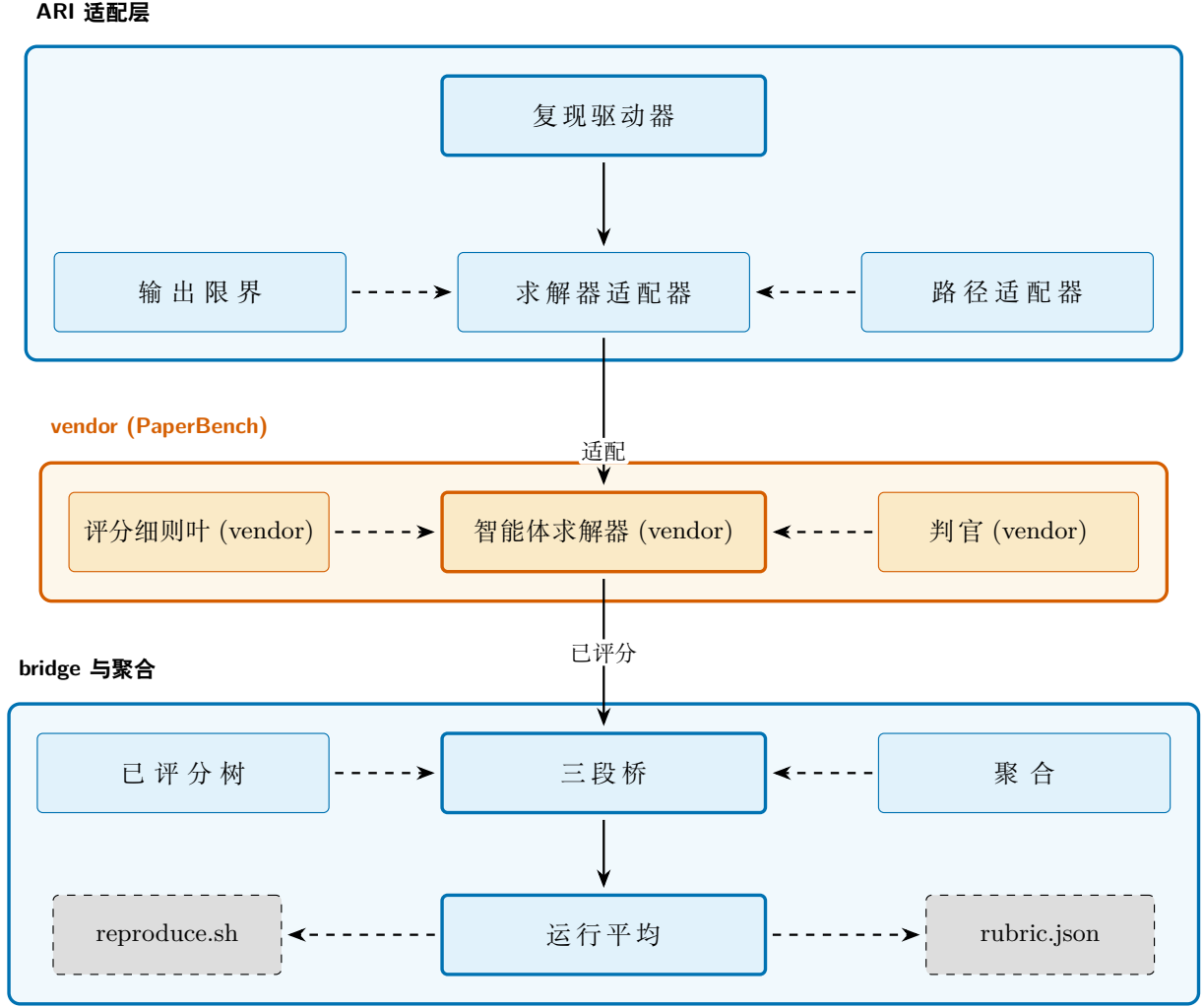


图 5: `paper-re` 技能的组件, 分三层。顶行为 ARI 的适配层 (复现驱动器、求解器适配器、输出限界、路径适配器); 中行为 vendor 化的 PaperBench 包 (智能体求解器、评分细则叶、判官); 底行为 bridge, 把按 submission 的已评分树折叠成单一 `rubric.json` 得分, 加上 ARI 侧的 `reproduce.sh`。详见 section 4.5。

动窗) 跟踪近期标签。对于其标签在该窗中相对最常见标签为欠表达的节点, 函数返回:

$$\text{div}(n) = \begin{cases} +0.05 & \text{cnt}(n) \leq \frac{1}{2} \text{cnt}_{\max}, \\ 0 & \text{否则,} \end{cases} \quad (4)$$

其中 $\text{cnt}(n)$ 为窗内 $\text{label}(n)$ 的出现次数, $\text{cnt}_{\max} = \max_{\ell} \text{cnt}(\ell)$ 。0.05 这个常数有点小: scientific score 仍主导排序, 奖励仅打破近似平手。

LLM 回退

若 LLM 返回无法解析的索引, `select_next_node` 通过 `_select_fallback` 辅助函数 (按 `_fallback_score` 式对候选排序) 回退至确定性排序。默认评分式为

$$\text{fb}(n) = r(n) + \text{div}(n), \quad (5)$$

其中 $r(n) \equiv \text{scientific_score}(n)$ 是节点的标量奖励 (section A)。`_select_fallback` 在存在

`has_real_data=True` 节点时优先选取它们, 然后返回 $\text{fb}(\cdot)$ 最大的候选。即使 LLM 调用退化, 循环也保证向前推进。

评估层的配置

sections 3.1 and 3.3 中出现的 4 个评估面可独立配置 (默认值复刻上式; 未修改的配置等同于 no-op)。

合成公式. 由 `evaluator.composite` 选择。 $\{a_k\} \rightarrow \text{scientific_score}$ 的合成方式可选: 加权调和平均 (默认)、加权算术平均、瓶颈式 `weighted_min`、加权几何平均。调和与几何平均使用 $\epsilon = 0.01$ 对零值轴下界, 保证分母有限。

前沿评分策略. 由 `bfts.frontier_score` 选择。eq. (5) 对应 `scientific_plus_diversity`; 其余可选 `scientific_only` (去除 div); `depth_penalized` 在 $\text{div}(n)$ 之上叠加 $-\lambda \text{depth}(n)$, 其中

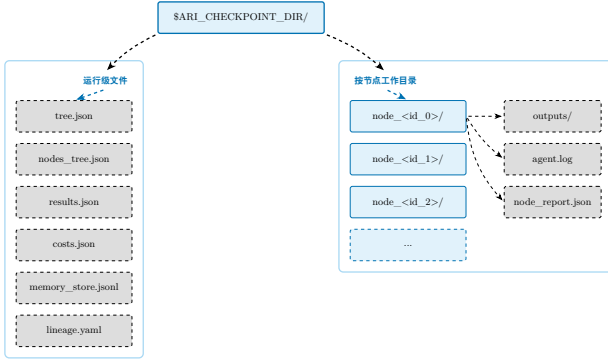


图 6: 检查点目录布局。运行级文件 (左带) 由编排器写入; 按节点工作目录 (右带) 保存一次 BFTS 扩展内产生的产物。复现一次运行就是「针对此目录重跑」, 该布局保证不需要其他状态。

$\lambda = \text{depth_penalty_lambda}$; ucb_like 在 $\text{div}(n)$ 之上叠加 $c_{\text{ucb}} \sqrt{\log N / (e(n) + 1)}$, 其中 $c_{\text{ucb}} = \text{ucb_c}$, $N = \sum_{n'} e(n') + |\mathcal{F}|$ 。

评估轴集合. 由 `evaluator.axis_mode` 选择。
`dynamic`(默认)= 通用 5 轴 floor + 有效 rubric + `idea.json` 的 plan-keyword 轴; `legacy` 固定 5 轴; `custom` 直接使用 $\{(name, description, w)\}$ 列表。

选择 prompt. 由 `bfts.select_prompt` 与 `bfts.expand_select_prompt` 设置。通过 `FilesystemPromptLoader` 键替换 `algorithm 1` 中两个 `selectLLM` 所用模板。选择模板须接受占位符 `experiment_goal`、`memory_context` 与 `candidates`; 扩展目标模板接受 `experiment_goal` 与 `candidates`。

3.4 扩展 (每次调用 1 个子节点)

`expand` 函数每次调用至多生成一个新子节点。同一父节点希望多个子节点的调用方反复调用 `expand`, 通过 `existing_children` 参数把已生成的子节点串入, 使 LLM 避免提议重复方向。

各新子的标签由 LLM 判断而非硬编码模板决定; 扩展提示 (逐字载录于 section B.1) 接收:

- 父节点状态 (`succeeded` / `failed` / `no-real-data`) 与 `_scientific_score`;
- 父节点结构化的 `node_report` (`delta_vs_parent` / `concerns` / `hints`, 由 `node-report` 构建器生成) 若存在, 以便规划者瞄准具体弱点;
- 同深度的兄弟得分, 以及自根至父的祖先得分;

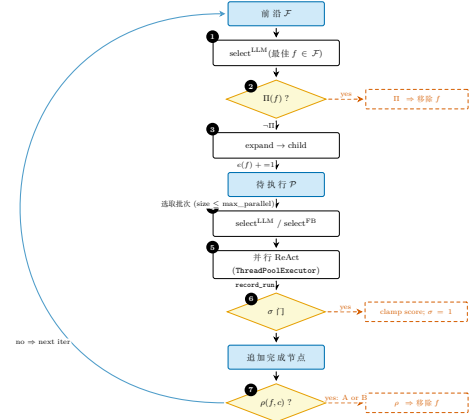


图 7: BFTS 单次外层迭代的状态机视图 (自上而下的单一流程图)。步骤 1–3 为内层扩展子循环; 步骤 4–7 为外层并行执行循环。黄色菱形为谓词门 (Π, σ, ρ); 右侧红色虚线框为各谓词触发的副作用 (前沿移除 / 分数夹紧 / retire)。图中未画出的辅助状态: 每父节点的扩展计数器 $e(\cdot)$ 在步骤 3 自增, 在步骤 7 的 ρ 中被使用; 标签历史 H (窗口 W) 在步骤 5 由 `record_run` 追加, 在步骤 4 用于计算多样性奖励 $\text{div}(\cdot)$ 。可与 fig. 4 (将相同循环呈现为带 LLM 提示的细粒度控制流主线) 对比。

- 整树多样性指标 (目前已见的唯一标签、深度分布); 以及
- 此父节点已生成的子节点标签, 避免重复提议。

子节点以 `open` 创建, 交给智能体循环执行 (section 3.7), 所有轴填齐后变为 `done`; 智能体循环抛异常时变为 `failed`。

3.5 剪枝与终止

`should_prune` 实现剪枝谓词 Π (eq. (1)): 总节点上限、深度上限与 `sterile` 标志。深度检查激活了 `max_depth` 配置。前沿退役谓词 ρ (eq. (3)) 在每个节点完成后施加: 规则 A 在子的奖励 $r(\cdot)$ 超过父 f 时退役 f , 规则 B 在 f 已被扩展 E_{max} 次时退役。 ρ 不删除节点, 其历史仍保留在 $\mathcal{T}$ 中, 只是收缩前沿池, 使搜索分散而不是无限挖掘同一父节点。

下列任一条件满足即停止:

- 全局 `max_total_nodes` 达到;
- 前沿中所有节点均触发 Π 且 `pending` 为空;
- 调用侧成本追踪器强制的步数 / 成本预算耗尽;
- 停滞检测器 `detect_stagnation` 观察到 `_scientific_score` 的运行最大值在滑动窗内不再改善;

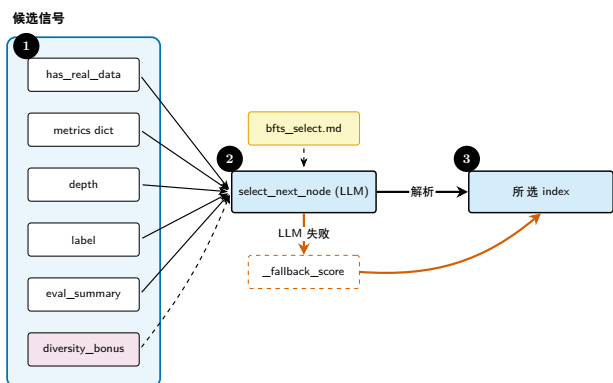


图 8: 交给 LLM 用以挑选下一节点的选择信号。输入被拼接成结构化候选描述；软性的 `diversity_bonus` 仅作平手解决，不作主信号。

- 谱系判断模块的 `LineageDecision` 显式终止该分支 (参见 section B.4)。

这些当中实际最常发生哪一种，是一个经验问题，留待冻结检查点就位后由 section 5 回答；本报告不主张其分布。

3.6 谱系与可复现性

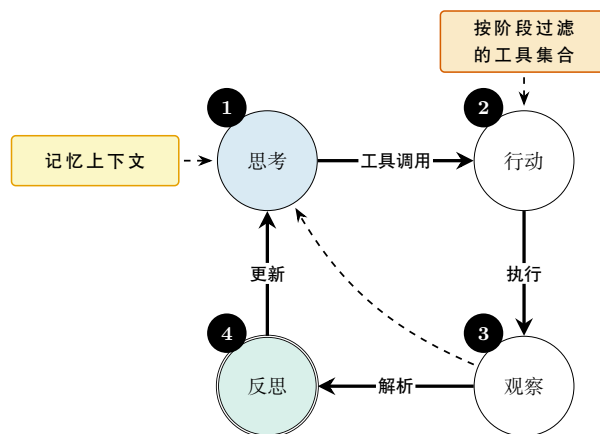
谱系即最佳节点祖先的链条，作为 `tree.json` 中的 `lineage` 键写入检查点。`LineageState` 数据类捕获 BFTS 用于决定是否继续的滚动统计；跨检查点的遍历器 `walk_ancestor_ckpts` 提供跨运行的父指针。构想池经 `get_idea_pool_for_ckpt` 继承，排序后的根构想选择记录在 `RootChoice` 数据类中以便事后审计 (提示词: section B.5)。

可复现性依赖三项不变量。其一，每次随机化操作以节点标识符播种。其二，每次工具调用将参数与输出写入所属节点的工作目录，绝不写入父节点——由 MCP 层的 `cow_node_id` 校验强制。其三，成本账本在 `CostTracker.init` 处为每节点附加美元金额，因此给定相同提示，预算检查保持确定。

3.7 ReAct 驱动器

每个节点的扩展在 `AgentLoop` 类中运行 ReAct 风格的智能体循环。最大步数由 `MAX_REACT_STEPS = 80` 控制，可按实例通过 `max_react_steps` 关键字覆盖。另一保护 `MIN_TOOL_CALLS = 2` 防止平凡过短的轨迹。智能体的系统提示词逐字载录于 section B.6。

智能体可用工具集合按阶段由工具管理器过滤；例如探索阶段屏蔽论文撰写工具，使 BFTS 扩展不会便捷地变成草稿。一小部分 MCP 工具为 `ari-core` 自身预留 (memory CoW 操作) 并对 LLM 隐藏；这保



`MAX_REACT_STEPS = 80, MIN_TOOL_CALLS = 2`

图 9: ReAct 循环。实线为主循环；虚线反向边表示当观测已闭合开放问题时的提前退出。

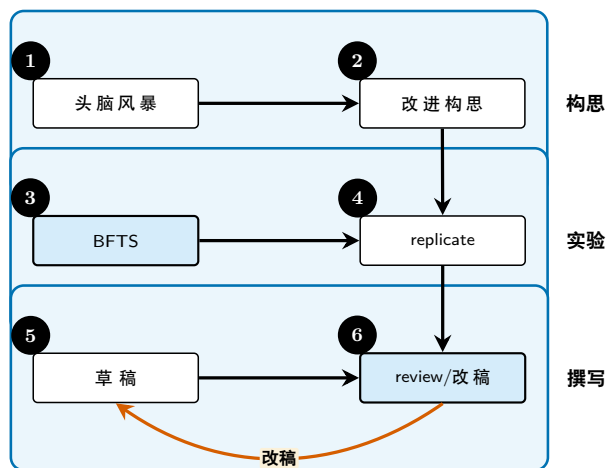


图 10: 论文生成泳道：构思、实验、撰写。每条泳道对应一个阶段；由 review 回到 draft 的反向边是唯一的非单调转移 (section 4.4)。

证模型无法设置任意 `node id` 来绕过记忆技能的写时复制校验。

4 论文生成流水线

4.1 流水线概述

探索终止后，存活谱系被交付给论文生成流水线——ARI 的第二棵活动树 (fig. 10)。撰写阶段将谱系汇编为草稿，评审阶段以评分细则对该草稿评分；两者由改稿循环 (section 4.4) 耦合。流水线从探索留下的按节点产物中，组装撰写器可依据的证据 (section 4.2)，并持久化三件输出： $\text{\LaTeX}$ 草稿、带理由的按叶评审，以及本次运行所用的评分细则实例 (评分细则可能逐论文生成而非预先固定，故保存)。

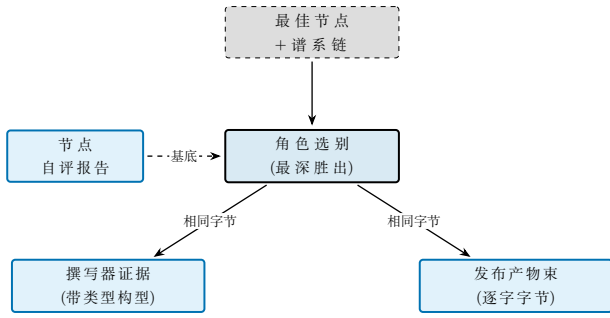


图 11: 证据组装。确定性组装器把最佳节点的谱系变为一次选择, 该选择以相同的选定字节供给两个消费者——撰写器的谱系证据与已发布产物束。节点自评报告 (内容哈希、成功自评、逐字构建/ 运行命令、所捕获硬件) 为选择奠基, 故每个贡献文件都由已记录元数据选出; 路径冲突时最深的出现胜出, 链上的删除则移除该文件。

4.2 证据组装

两棵树之间坐落着一个确定性的证据组装器 (fig. 11)。它决定谱系的按节点产物中撰写器可依据者, 并将其重塑为单一的面向科学的记录。该设计使这道接缝可审计——某一主张可追溯到哪个节点, 是已记录元数据的函数, 而非语言模型的判断——从而使 section 4.7 的护栏有具体之物可强制执行。

选别按角色进行: 一个节点可以不同方式贡献给三个消费者, 由单一谓词族判定各自。节点在携带真实测量 (或评估器认定其运行成功) 时进入合成集合; 在相对于父节点新增或修改了源文件时进入代码集合; 在其步骤成功时进入叙事集合。仅探索了死胡同的节点不进入任何集合。每条准则都是已记录元数据上的纯谓词, 故相同输入总是选出相同的贡献集合。

源文件的收集方式是: 沿最佳节点 n^* 的祖先链 $\text{lin}(n^*)$, 按深度顺序叠覆每个贡献节点触及的文件。由于子节点在父节点工作树的副本内执行, 一条相对路径可能出现于多个深度; 最深的出现胜出, 链上的删除则移除该文件——于是重建集合与最佳节点的实际工作树一致。该选择以相同字节供给两个消费者: 撰写器的谱系证据与已发布产物束, 因此论文据其描述的谱系代码即为所发布的代码。选择为链上专属——最佳谱系之外的节点即便其测量被合成集合报告也不发布, 并记入 provenance——此外为伪代码忠实性, 撰写器还另获最高分节点的原始源码, 但其本身并非发布集合。

组装器直接从每个节点的工作树读取代码、参数与测量值, 而非读取摘要。当实验声明了带类型的结果时, 记录将输入参数 (问题规模、线程数、随机种子等配置旋钮) 与测量输出 (吞吐、精度、延迟) 及派生量 (效率、模型预测上限) 分离。此分离并非装饰: 跨

构型报告的按键极值被确定性地、且仅在测量键上归约,

$$\text{best}(m) = \underset{c \in \mathcal{E}}{\text{red}} c[m], \quad m \notin \mathcal{I}, \quad (6)$$

其中 red 按该指标声明的方向取最大或最小, $\mathcal{E}$ 为贡献构型的集合, $\mathcal{I}$ 为从一切归约中剔除的已声明输入键集合。剔除输入正是防止把巨大的输入规模——比如数百万非零元的问题规模——误读为主结果。语言模型被限定于真正需要它的唯一任务: 将逐字源码与日志转化为散文与伪代码, 并叙述方法论。它从不提供数值。

支撑这一切的是节点完成时写出的结构化自评报告 (检查点内的 `node_report.json`)。它以内容哈希记录: 该节点相对父节点新增或修改了哪些文件; 节点自身的成功自评连同评估器标记的关切; 逐字的构建与运行命令; 以及测量所运行的硬件。哈希使源选择可复现, 并使已发布产物束能携带由所记录命令构建的复现脚本; 所捕获的硬件让论文的环境章节能够基于事实撰写, 而非留作“未记录”。

4.3 评分细则形式

我们将评分细则 R 建模为有限树, 叶节点携带元组 $(c_i, w_i, \text{judge}_i)$ 。每个叶 i 含类别描述符 c_i 、非负权重 w_i 满足 $\sum_i w_i = 1$, 以及返回分级得分的判官函数 $\text{judge}_i : P \rightarrow [0, 1]$ 。论文得分即凸组合

$$\text{score}(P) = \sum_i w_i \text{judge}_i(P). \quad (7)$$

此树结构评分细则与 PaperBench 共享, ARI 通过 vendor 化的包复用之 (section 4.5), 使其评分与聚合语义逐字追随上游。判官有两族并存: 面向定性叶的 LLM 评分器, 以及面向具二元或数值校验叶的确定性评分器 (如「成本是否以美元绝对值报告?」)。两者满足同一 Evaluator 接口, 因此搜索引擎 (section 3) 与论文流水线通过一个接缝将轴归约为标量。

评分细则以两种模式之一生成。agent-benchmark 模式按论文的科学贡献分解树, 各叶询问候选 submission 的执行输出是否再现论文——即原始 PaperBench 范式。paper-audit 模式中树的顶层子节点是固定的再现性轴集合, 各叶评分论文文本自身的描述完整性——面向 HPC 再现性审计的范式倒置, 问题不再是「智能体能否再现这篇论文?」而是「这篇论文是否描述了足够再现所需的信息?」。venue 知识 (用哪些轴、如何措辞) 由逐 venue 模板提供而非焊入引擎, 故 ARI 核心保持 domain-agnostic (P4), 新增 venue 是数据变更而非代码变更。

将 paper-audit 得分从退化区域——空 submission 把所有 submission 主语的叶推向「否」——

提升出来, 需要把叶的问题改为以论文而非 (缺失的)submission 为主语提问、把论文的图与正文一并喂给判官 (使具视觉能力的模型可加以利用)、并允许将可选的 Artifact Description / Artifact Evaluation 附录作为额外输入。这些调整完全留在 ARI 的包装层, vendor 化的判官无改动, 从而使跨上游版本的得分保持可比。我们有意不从论文自身报告值合成再现日志: 那会短路 executed-submission 阶段, 以与 leaderboard 运行不可比的方式抬高得分。

4.4 评审/改稿循环

改稿循环按评审反馈 $J(P_t)$ 迭代地将 P_t 重写为 P_{t+1} :

$$P_{t+1} = \text{Refine}(P_t, J(P_t)). \quad (8)$$

当评审者接受草稿——建模为收敛标准 $\text{score}(P_{t+1}) - \text{score}(P_t) < \epsilon$ ——或达到逐节的小幅改稿上限时, 循环终止。哪条分支更常触发是一个经验问题, 待冻结检查点就位后由 section 5 回答; 此处不做定量主张。

改稿步骤在每条轴上有意非单调: 修复清晰度问题可能轻微降低某数值轴。这正是 fig. 10 中反向边的来源, 也是收敛标准定义在聚合得分而非逐轴的原因——并配以逐轴下限 (section 4.7), 以防聚合值掩盖单一轴的崩溃。

4.5 PaperBench 集成

复现路径不是对 PaperBench 的重新实现: 上游包被无改动 vendor 化, 使 ARI 逐字追随其任务与评分细则语义, ARI 仅在其周围提供一层薄包装。包装将协议忠实的循环公开为三个共享词汇的可组合阶段 (fig. 12):

1. *rollout* ——复现智能体从论文出发走向 submission (代码, 以及可选的再现脚本);
2. *reproduce* ——在隔离容器中、于本地机器上或派发到集群执行 submission;
3. *grade* ——vendor 化的判官按评分细则树为 submission 评分。

两个设计选择使其忠于基准。其一, 评分交由 vendor 化判官, ARI 的适配器只整形其输出并把重复运行聚合为单一得分, 故升级 PaperBench 是不改变叶评分结合方式的 vendor swap——同一包络流入 eq. (7), 二元叶取 $\text{judge}_i \in \{0, 1\}$ 。其二, 包装 *fail loud*: 当无法提供基准公平的运行时 (无容器运行时、无调度器、无 GPU 资源登记), 抛出异常而非静默降级到更弱的

宿主, 因为静默降级会使得分与 leaderboard 条目不可比。旧的 *silent-fallback* 行为仅经显式 opt-in 可用。

第二个关注点是让复现智能体对宿主机实际提供的能力保持诚实。若放任上游提示词, 智能体倾向于 scaffold 出调用宿主机不存在的二进制的再现脚本, 并不论文真实语言而默认使用 Python。ARI 通过诱导智能体在 scaffold 前先 probe 环境、并注入由可用工具链/GPU/网络可达性的实时 probe 生成的逐宿主主机注册块加以打消——使智能体读到的与宿主机实际能做的相一致。

4.6 领域广度: 执行配置契约

PaperBench 上游假定单 GPU 的单容器。为覆盖方法论本质上并行的论文——多 rank MPI 内核、多节点训练、集群特定的模块环境——ARI 以可选的执行配置 (fig. 13) 扩展评分细则。该配置 domain-agnostic: 它命名的是并行执行形态 (单 CPU、单/多 GPU、MPI、MPI+GPU) 连同论文的规模包络与调度器提示, 而非论文的科学领域。当论文为单机时配置省略, 流水线退化为原始单节点调用: 契约是加法的, 而非破坏性的。

配置供给两个消费方 (fig. 13)。它作为简洁的「计算节点执行约定」块——共享文件系统纪律、优先用调度器的启动器而非裸 MPI 启动器、环境激活、wall-clock 包裹——附加到智能体提示词, 并参数化集群派发 (节点、GPU、内存与放置请求)。由于各集群接受的调度器标志不一致, 调度器在运行时 probe 本地调度器并丢弃其不支持的标志。如此同一份评分细则在多 GPU 集群、无 GPU 登记的沙箱分区、或单一工作站上都能忠实派发。回归测试强制其逆命题: 运行于无关集群分配内的 ARI 绝不可把集群/MPI 特定的文字泄漏进非 HPC 论文的提示词。仅此包装层变化; vendor 化的基准与 ARI 核心在该扩展中保持不变。

4.7 故障模式与护栏

四种故障模式频繁到值得命名, 各自配以结构性——而非仅建议性——的缓解:

- **仅 LLM 支撑**: 论文断言一项谱系中无任何节点支撑的主张。缓解策略将撰写器约束到预先选定的谱系产物集合, 使每项数值或因果主张都可追溯到产生它的节点; 草稿无法引用未被生成的证据。
- **引用幻觉**: 论文引用不存在的文献。缓解是结构性的——撰写器不得发明引用键, 且每条 bib 条

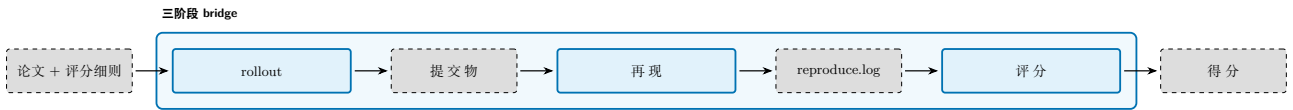


图 12: 将 PaperBench 复现路径表示为三个可组合阶段。智能体的 *rollout* 把论文及其评分细则转化为 submission; *reproduce* 在隔离沙箱中执行该 submission, 捕获 `reproduce.log` 及其输出; *grade* 按评分细则树为 submission 评分。ARI 的三阶段 bridge 统筹这些阶段, 并把重复运行聚合为单一得分。

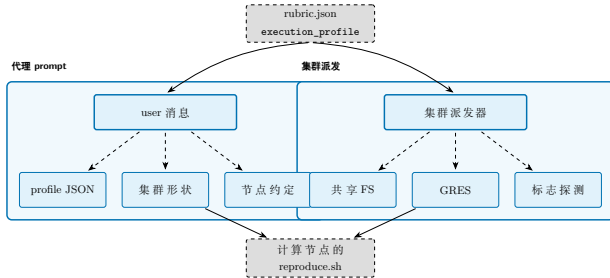


图 13: 执行配置契约从单一评分细则产物流入两个消费方: 复现智能体的提示词 (左) 与集群调度器 (右)。运行时 probe 使派发的资源请求适配本地调度器, 因此一份评分细则在异构集群上无需修改即可忠实运行。

目在被采纳前都对外部索引校验 (section 6)。

- **改稿漂移:** 聚合得分上升而论文丧失连贯性。缓解是逐轴下限: 任一轴跌破阈值即中止改稿, 使一轴的提升无法掩盖另一轴的崩溃。
- **数值漂移:** 报告的量与其所概括的已记录结果不一致。由于每项数值主张都指明其所依据的节点与测量值 (section 4.2), 每项都从已记录结果确定性地重新推导并在容差内比对; 与重新计算值不一致的值会被标记。默认情况下闸门仅报告此类不一致而不阻断; 仅在更严格的审计策略下, 最终检查中未消解的不一致才会被拒绝而非发表。

第一道护栏也限制 context: 撰写器获得选定谱系产物与最高分节点的源码, 且受限於大小预算, 因此即便长时间运行草稿也保持在模型的 context 窗口之内。

5 初步观察

下文的观察是稼动中的系统在此阶段所确立的内容。它们属于初步性质; 就端到端再现的情形而言, 这是单次运行而非受控基准; 面向面板规模的定量研究留待今后工作。本报告别处的定量图表仅为示意, 均由固定种子的样本数据生成, 以保证文档构建可复现。

有三项性质直接由系统的构造成立。成本核算是精确的: 每次模型调用都被附加美元成本, 并按运行汇总进检查点的结果记录。探索循环经停滞检测器

(section 3.5) 终止, 而非仅由步数或成本预算 ($T_{\max} / C_{\max}$) 触发。且执行配置路径按规约行事 (section 4.6): 评分细则生成对 HPC 论文发出携带论文所报告规模包络的多 rank (MPI) 执行配置, 并对单机论文省略; 而一次多标志集群调度, 在运行时 probe 剥离该调度器不支持的标志之后, 被本地调度器接受。

端到端地, v0.8.0 流水线在一个非可逆压缩对象——一个 GPU 上的误差界限受控科学数据压缩器——上得到执行。此单一对象上达成的再现程度虽属部分但相当扎实。智能体获取了论文所引用仿真数据集的真实切片, 而非以合成数据替代; 编译了原生 CUDA 插值内核并在 GPU 上运行; 实现了带逐层自动调优的 Lorenzo 与多级插值预测器; 并扫掠了一段误差界范围, 以相应的压缩比恢复出大约 50 至 90 dB PSNR 的重建质量。对照细粒度的评分细则, 该次运行仅通过了约十分之一量级的加权叶节点, 其原因有两点, 皆根植于智能体的行为: 被评分的指标来自一个 CPU 预测器——GPU 内核确已构建并执行, 但智能体将其预测质量从所报告的结果中省略——且智能体在仅消耗其时间预算的一小部分后便自行终止, 将其余数据集与专有的无损阶段留待未试。

受控评估——在冻结检查点上的中位运行成本、按种子的探索曲线、按类别的评分细则分布, 以及在一组对照性目标论文面板上的完整 rollout 复现得分——留待今后工作。

6 相关工作

我们将 ARI 与四条先行研究脉络对照定位。

6.1 面向代码的树搜索

AIDE [2] 将 ML 工程形式化为在计划/代码/调试状态上的树搜索; AlphaCode [3] 是大规模生成与过滤在竞赛编程上的范例; MLE-bench 评估器 [4] 现已成为衡量 ML 工程智能体的标准。ARI 沿用 AIDE 的 BFTS 骨架, 但增加 (i) LLM 主导的候选选择器 (section 3.3) ——不固定闭式标量效用, 使选择提示可联合权衡 `has_real_data`、整套 metrics、深度与

软性 `diversity_bonus`; 以及 (ii) 跨运行的谱系 (section 3.6), AIDE 与 MLE-bench 均未形式化此项。

6.2 自主研究智能体

AI Scientist 系列 [5, 6] 端到端闭合类似循环 (idea $\rightarrow$ experiment $\rightarrow$ paper), 但未将评分细则与改稿循环作为一等对象暴露。VirSci [7] 模拟多个科学家人格, 以共识评分输出; Agent Laboratory [8] 将智能体视为协作者而非驱动者。ARI 在两点运维上有别: 一切都驻留在检查点中 (P1 / P4), 且评分细则即探索与撰写之间的契约。

6.3 论文评估

PaperBench [9] 是与我们 `paper-re` 集成最接近的对应物; 它开创了评分细则即树的形式。我们沿用其按叶的分级与聚合权重 (eq. (7)), 并扩展以按轴下限作为改稿护栏 (section 4.7)。

6.4 基础

智能体循环遵循 ReAct 模式 [1]; 改稿循环源自 Self-Refine [10] 与 Reflexion [11]; 在思考步给 LLM 一份逐步草稿板的设计源自 Chain-of-Thought [12]。这些参考文献均未提议检查点作用域的状态, 这是 ARI 的贡献。

早期草稿曾另引一份 ML 研究智能体基准, 但当前候选来源均无法通过我们的校验规则 (section 4.7) 时, 我们将该引用从相关工作中移除; 此节仅保留可经 S2 校验的条目。

7 讨论与局限

7.1 确定性 vs 新颖性

P2, 确定性原则, 是系统最具约束力者。严格执行 P2 迫使 *ari-skill-memory* 声明“无 LLM 调用”——它必须以确定性关键字函数对检索打分, 而非依赖输出会随远端服务漂移的嵌入模型。代价是检索质量受关键字评分能力上界约束。

这并非偶然: 每一项可复现性的胜利都以一道关闭非确定性的门为代价。我们接受此代价, 因为契约对象是检查点, 而非运行。

7.2 扩展极限

支撑 BFTS 运行循环的单机假设是最大的扩展约束。希望并行扩展 $b > 10$ 的运行需要不同的并发层;

目前 BFTS CLI 循环中的单个 `ThreadPoolExecutor` 是唯一的工作池。多机扩展需要将谱系纪律 (P4) 推广至支持合并, 而不仅是祖先遍历。

另一约束是 LLM 评分器的成本。经验上, 评分器与探索器并列为占主导的开销项之一; 它在绝对美元上是否位列第二, 需待冻结检查点就位后由 section 5 回答。按 (论文哈希, 细则哈希) 缓存评分器输出会有所帮助; 此特性尚未发布。

Algorithm 1 BFTS run-loop(单次外层迭代)。

Input: 状态 $S = (\mathcal{F}, \mathcal{P}, e, H)$; 配置 $(B_N, B_D, E_{\max})$ 与 worker 上限 $\text{max_parallel} = \min(\text{max_parallel_nodes}, 4)$

Output: 更新后状态 S'

```
1:  $\triangleright$  — 内层扩展子循环 —
2: while  $\mathcal{F} \neq \emptyset \wedge |\mathcal{P}| < \text{max\_parallel} \wedge |\mathcal{N}| < B_N$  do
3:    $f \leftarrow \text{select}^{\text{LLM}}(\mathcal{F})$   $\triangleright \text{select\_best\_to\_expand}$ 
4:   if  $\Pi(f; |\mathcal{N}|)$  then
5:      $\mathcal{F} \leftarrow \mathcal{F} \setminus \{f\}$ ; continue
6:   end if
7:    $c \leftarrow \text{expand}(f, \text{depth\_note}, \text{budget\_note})$ 
8:    $\mathcal{P} \leftarrow \mathcal{P} \cup \{c\}$ ;  $e(f) + = 1$ 
9: end while
10:  $\triangleright$  — 外层并行执行批次 —
11:  $B \leftarrow \emptyset$ 
12: while  $|B| < \min(\text{max\_parallel}, |\mathcal{P}|)$  do
13:    $n \leftarrow \text{select}^{\text{LLM}}(\mathcal{P})$   $\triangleright \text{select\_next\_node}$ : 单个节点
14:    $\mathcal{P} \leftarrow \mathcal{P} \setminus \{n\}$ ;  $B \leftarrow B \cup \{n\}$ 
15: end while
16: for all  $n \in B$  并行 do
17:    $n' \leftarrow \text{ReAct}(n)$   $\triangleright \text{AgentLoop.run}$ , 可能失败
18:    $H \leftarrow (H \parallel \text{label}(n'))[-W:]$   $\triangleright \text{record\_run}$ 
19:    $\mathcal{F} \leftarrow \mathcal{F} \cup \{n'\}$ 
20:   for all  $p \in \mathcal{F} : p = \text{pa}(n')$  do  $\triangleright$  规则 A
21:     if  $r(n') > r(p)$  then
22:        $\mathcal{F} \leftarrow \mathcal{F} \setminus \{p\}$ 
23:     end if
24:   end for
25:   for all  $p \in \mathcal{F}$  do  $\triangleright$  规则 B
26:     if  $e(p) \geq E_{\max}$  then
27:        $\mathcal{F} \leftarrow \mathcal{F} \setminus \{p\}$ 
28:     end if
29:   end for
30: end for
31: return  $S' = (\mathcal{F}, \mathcal{P}, e, H)$ 
```

Algorithm 2 证据组装 (探索 $\rightarrow$ 撰写器输入)。

Input: 带按节点报告的谱系节点 $\mathcal{N}$; 指标方向映射

Output: 科学记录 $\mathcal{E}$: 带类型构型、按键极值、方法论叙事

```
1:  $n^* \leftarrow \arg \max_{n \in \mathcal{N}} r(n)$   $\triangleright$  同分: 已验证, 其次更深
2:  $L \leftarrow \text{lin}(n^*)$   $\triangleright$  根到最佳的祖先链
3:  $\mathcal{C} \leftarrow \{n \in L : \text{CONTRIBUTES}(n)\}$   $\triangleright$  角色谓词 (代码)
4:  $F \leftarrow \emptyset$   $\triangleright$  相对路径  $\mapsto$  (节点, 深度)
5: for all  $n \in L$  按深度顺序 do
6:   for all  $n \in \mathcal{C}$  新增/修改的路径  $p$  do
7:     if  $p \notin F \vee \text{depth}(n) \geq \text{depth}(F[p])$  then
8:        $F[p] \leftarrow (n, \text{depth}(n))$   $\triangleright$  最深胜出
9:     end if
10:   end for
11:   for all  $n$  删除的路径  $p$  do
12:     从  $F$  移除  $p$   $\triangleright$  删除以叠覆移除
13:   end for
14: end for
15: 在尺寸预算下逐字读取每个  $p \in F$  的字节
16: for all 测量键  $m \notin \mathcal{I}$  do
17:    $\text{best}(m) \leftarrow \text{red}_c c[m]$   $\triangleright$  eq. (6); 确定性
18: end for
19:  $\text{NARRATE}(F) \triangleright \text{LLM}$ : 逐字源码/日志  $\rightarrow$  散文、伪代码
20: return  $\mathcal{E}$ 
```

A 记号表

本附录汇集正文中使用的全部记号及其含义。

符号	含义
$\mathcal{N}$	搜索树的节点集
$\mathcal{T}$	搜索树 $(\mathcal{N}, E)$
$n \in \mathcal{N}$	单一节点
$\text{ch}(n)$	n 的子集
$\text{pa}(n)$	n 的父节点
$\text{depth}(n)$	n 的深度
$s(n)$	节点状态 $\in \{\text{open}, \text{eval}, \text{done}, \text{failed}\}$
$r(n)$	n 的标量奖励
$\mathbf{e}(n)$	按轴评估向量
ϕ	轴 $\rightarrow$ 标量聚合器
$\text{div}(n)$	多样性奖励 (eq. (4))
$\text{fb}(n)$	确定性回退排序 (eq. (5))
$T_{\max}, C_{\max}$	步数 / 成本预算
R	评分细则树
c_i	第 i 叶的类别
w_i	第 i 叶的权重
judge_i	第 i 叶的判官
P	论文草稿
$\text{score}(P)$	论文聚合得分 (eq. (7))
Refine	改稿算子 (eq. (8))
ckpt	检查点目录
$\text{lin}(n)$	终止于 n 的谱系链

B 运行时 LLM 提示词

本附录列出 ARI 在运行时所用的 LLM 提示词, 逐字摘自 v0.8.0 时刻的 `ari-core/ari/prompts/`。每段清单与磁盘源码逐字节一致。因为输入模型的字节本身决定其行为, 提示词不进行翻译, 在本报告的各语言版本中均以原文原样呈现。

编排器

B.1 BFTS 扩展

```
You are expanding a BFTS research tree node.

{goal_line}Parent node id={parent_id_short}, depth={parent_depth}, status={parent_status}
{depth_note}{budget_note}Parent metrics: {parent_metrics_json}
Parent summary: {parent_summary}
{sci_note}{idea_block}{parent_report_block}
{siblings_block}{ancestors_block}{existing_block}{diversity_block}Propose exactly ONE child research direction
that is the most scientifically valuable next step. For the "label" field, strongly prefer one of the canonical
labels [draft, improve, debug, ablation, validation]; only invent a custom label (e.g. "replication",
"generalization") when none of the five fits meaningfully — the custom string will be preserved as raw_label.
Base your choice on the experimental context above, not on a fixed template.
```

Label selection guidance:

- 'debug': parent FAILED or has no real data — diagnose and fix it.
- 'improve': parent succeeded and you want to push its metrics higher by tuning parameters, flags, or algorithms.
- 'ablation': isolate which component drives the parent's gains by removing or varying ONE component. State explicitly what is removed/varied and what delta vs. the parent metrics you expect.
- 'validation': rigorously verify the parent's claims (different seeds, edge cases, stress tests, expected-degradation checks).
- 'draft': start a fresh implementation from scratch to introduce a fundamentally NEW perspective (use this instead of inventing a new label like 'replication' or 'generalization').

Reply ONLY with a JSON array containing exactly one element: [{"label": "<one of: draft|improve|debug|ablation|validation>", "direction": "..."}]

Example: [{"label": "validation", "direction": "<one-sentence plan>"}]

B.2 BFTS 扩展期选择

You are selecting which completed research node to expand next in a BFTS tree.

Experiment goal: {experiment_goal}

Completed nodes awaiting expansion:
{candidates}

Select the single most promising node to expand. Prefer nodes with high scientific_score, strong metrics, and unexplored directions. Avoid nodes that have already been retried many times or are at excessive depth. Reply with ONLY the index number (0-based).

B.3 BFTS 选择

You are selecting the most promising node to explore next in a research tree.

Experiment goal: {experiment_goal}

Relevant past memories:
{memory_context}

Candidates:
{candidates}

Selection criteria:

- Nodes with has_real_data=True and strong metrics are high-value
- Consider all metrics holistically (multi-objective)
- Deeper nodes with excellent results are worth continuing
- A small diversity_bonus is awarded to underrepresented exploration types; treat it as a soft tiebreaker, not a primary signal

Reply with ONLY the index number (0-based) of the best node.

B.4 谱系统延判断

You are a research orchestrator. Given the current state of an exploratory research run, decide the next lineage-level action. Possible actions:

- | | |
|----------------|--|
| continue | — current idea is still productive; keep exploring |
| switch_to_idea | — current idea has stagnated; switch to an alternative |
| fanout | — current idea succeeded; explore an alternative in parallel |
| terminate | — research thread is exhausted; stop |

Choose the LEAST disruptive action that fits the evidence. Prefer continue unless there is a concrete reason to escalate. When choosing switch_to_idea or fanout, set target_idea_index to a value that appears in the alternatives pool. When choosing switch_to_idea, set disable_generate_ideas=true (the child should run with the chosen idea pinned, not regenerate). For fanout you may set it false so the child can also explore additional novel directions.

Reply ONLY with JSON:

```
{"action":str,"target_idea_index":int|null,"disable_generate_ideas":bool,"rationale":str}. No markdown.
```

B.5 根创意选择

You are a research orchestrator picking the ROOT idea for a run from a VirSci-generated pool. VirSci has already scored each idea by novelty/feasibility/clarity, but you have additional context (venue rubric, ancestor research thread, run notes). Your job is to pick the idea most likely to produce a strong submission for this venue, considering all signals.

Rules:

- chosen_index MUST be an integer that exists in the pool.
- Default to VirSci's choice (index 0) UNLESS another idea is clearly stronger given the additional context.
- One sentence rationale describing your reasoning.

Reply ONLY in JSON: {"chosen_index": int, "rationale": str}. No markdown.

智能体循环

B.6 ReAct 智能体系统提示

You are a research agent. You MUST use tools to execute experiments. Do NOT write plans or text descriptions — call a tool immediately.

AVAILABLE TOOLS:

{tool_desc}

RULES:

- Your FIRST action must be a tool call. Never output a text plan.
- If `make\_metric\_spec` tool is available and this is a new experiment (not a continuation), call it early to self-determine evaluation criteria.
- NEVER fabricate numeric values — only report values from actual tool outputs
- AFTER your final measurement run, when `emit\_results` is available, call it once to record a typed split between INPUT parameters (matrix size, thread count, seeds — knobs you ran on) and MEASUREMENTS (throughput, accuracy, latency — what you measured). This lets downstream stages distinguish "what we ran on" from "what we measured" so a best-of reduction never picks an input size as the result. Do NOT include input parameters in `measurements` and do NOT include measured outputs in `params`.
- When all experiments are done, return JSON: {"status": "success", "metrics": {...}, "summary": "..."}.
- Do NOT call gap_analysis or generate_hypothesis
- Ensure your experiment is reproducible: capture whatever information would be needed for an independent researcher to reproduce your results and verify your findings{memory_rules}{extra}

评估器

B.7 同行评议评估器

You are a research data extractor AND a scientific peer reviewer.

Analyze the experiment artifacts and return a JSON with:

has_real_data: bool (true only if numeric measurements appear in artifacts)

params: dict of INPUT/configuration values (NOT measurements). Examples: matrix size, thread count, seed.

measurements: dict of MEASURED quantities (the experiment's output). Examples: GFLOP_per_s, accuracy, latency.

```
metrics: dict — flat union of params and measurements (back-compat).
reason: str (one sentence describing what was measured)
{axes_block}
comparison_found: bool (true if results involve comparison with existing approaches)
```

When scoring `claim_implementation_alignment`, look for concrete preconditions or contracts the plan / model states (e.g. 'eliminates RFO traffic', 'preserves equivariance', 'requires sorted input') and cross-reference them against the artifacts. Score low when the implementation contradicts a stated assumption, even if the kernel runs without errors.

Return ONLY valid JSON, no markdown fences.

B.8 指标抽取

You are a research data extractor AND a scientific peer reviewer.

Analyze the experiment artifacts and return a JSON with:

```
has_real_data: bool (true only if numeric measurements appear in artifacts)
params: dict of INPUT/configuration values the experiment ran on. These are knobs, not outcomes. Examples:
matrix dimensions (M, K, nnz), thread count, batch size, ISA flags, seeds. They are NOT candidates for a
best-of reduction.
measurements: dict of MEASURED quantities — what a peer reviewer would treat as the experiment's result.
Examples: GFLOP_per_s, GB_per_s, latency_s, accuracy. ARE candidates for best-of.
metrics: dict — for back-compat, the union of params and measurements as a single flat dict (e.g.
{"GFLOP_per_s": 754.8, "M": 120000}). Downstream consumers may read either the typed split above or this
flat view. NEVER include the same key in both views with different values.
reason: str (one sentence describing what was measured)
axis_scores: dict with EXACTLY these five keys, each a float in 0.0-1.0:
- measurement_validity: are numeric measurements present and methodologically sound?
- comparative_rigor: are results compared against baselines or prior work?
- novelty: does the work advance beyond existing approaches?
- reproducibility: is there enough detail for someone else to reproduce the result?
- clarity_of_contribution: is the scientific claim stated clearly and specifically?
Score 0.0 on an axis when that dimension is absent or fatally weak. Score toward 1.0 as the dimension
approaches publishable quality. These axes are combined via weighted harmonic mean, so a low score on
ANY axis drags the overall score down — differentiate axes deliberately.
axis_rationales: dict with the same five keys; each value is one sentence explaining the score for that
axis.
comparison_found: bool (true if results involve comparison with existing approaches)
```

Return ONLY valid JSON, no markdown fences.

流水线

B.9 关键词管理员

You are a research librarian. Given experiment summaries, produce a SHORT broad academic search query (3-6 words) for Semantic Scholar. Use GENERAL terms (e.g. 'algorithm optimization', not 'custom 4-stage pipelined batch-norm fused kernel'). Avoid domain-specific jargon, acronyms, or overly narrow terms. Return ONLY the query string, no explanation.

向导 (Viz)

B.10 向导目标对话

You are an assistant helping a researcher set up an ARI experiment. ARI automatically writes, runs, benchmarks code, then produces a paper. Ask focused questions ONE AT A TIME to understand: (1) what to optimize or investigate, (2) how to measure success (metric name and direction), (3) platform/hardware constraints, (4) any baseline to compare against. Be concise. After 3-5 exchanges and sufficient info, output EXACTLY: ---READY--- followed by a complete experiment.md with sections: ## Research Goal, ## Evaluation Metric, ## Constraints. Do NOT output the MD before getting at least 2 user responses.

B.11 向导配置生成器

You are helping a researcher set up an automated experiment. Convert the following research goal into a concise experiment.md file with a ## Research Goal section. Keep it to 3-5 sentences maximum. Do not add code or technical details. Research goal: {goal}

References

- [1] Shunyu Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. 2023. arXiv: 2210.03629 [cs.CL]. URL: <https://arxiv.org/abs/2210.03629>.
- [2] Zhengyao Jiang et al. *AIDE: AI-Driven Exploration in the Space of Code*. 2025. arXiv: 2502.13138 [cs.AI]. URL: <https://arxiv.org/abs/2502.13138>.
- [3] Yujia Li et al. *Competition-Level Code Generation with AlphaCode*. 2022. DOI: 10.1126/science.abq1158. arXiv: 2203.07814 [cs.PL]. URL: <https://arxiv.org/abs/2203.07814>.
- [4] Jun Shern Chan et al. *MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering*. 2024. DOI: 10.48550/arXiv.2410.07095. arXiv: 2410.07095 [cs.LG]. URL: <https://arxiv.org/abs/2410.07095>.
- [5] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. *The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery*. 2024. arXiv: 2408.06292 [cs.AI]. URL: <https://arxiv.org/abs/2408.06292>.
- [6] Chris Lu et al. “Towards End-to-End Automation of AI Research”. In: *Nature* 651.8107 (2026), pp. 914–919. DOI: 10.1038/s41586-026-10265-5. URL: <https://doi.org/10.1038/s41586-026-10265-5>.
- [7] Haoyang Su et al. *Many Heads Are Better Than One: Improved Scientific Idea Generation by a LLM-Based Multi-Agent System*. 2024. DOI: 10.18653/v1/2025.acl-long.1368. arXiv: 2410.09403 [cs.CL]. URL: <https://arxiv.org/abs/2410.09403>.
- [8] Samuel Schmidgall et al. *Agent Laboratory: Using LLM Agents as Research Assistants*. 2025. DOI: 10.18653/v1/2025.findings-emnlp.320. arXiv: 2501.04227 [cs.AI]. URL: <https://arxiv.org/abs/2501.04227>.
- [9] Giulio Starace et al. *PaperBench: Evaluating AI’s Ability to Replicate AI Research*. 2025. DOI: 10.48550/arXiv.2504.01848. arXiv: 2504.01848 [cs.AI]. URL: <https://arxiv.org/abs/2504.01848>.
- [10] Aman Madaan et al. *Self-Refine: Iterative Refinement with Self-Feedback*. 2023. DOI: 10.48550/arXiv.2303.17651. arXiv: 2303.17651 [cs.CL]. URL: <https://arxiv.org/abs/2303.17651>.
- [11] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. *Reflexion: Language Agents with Verbal Reinforcement Learning*. 2023. arXiv: 2303.11366 [cs.AI]. URL: <https://arxiv.org/abs/2303.11366>.
- [12] Jason Wei et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2022. arXiv: 2201.11903 [cs.CL]. URL: <https://arxiv.org/abs/2201.11903>.